

RELATIONAL SEPARATION LOGIC FOR COMPILER VERIFICATION

MPRI Internship Report

Gabriel DESFRENE

Supervised by Xavier LEROY and François POTTIER

General Context

Modern software relies on high-level languages and formal verification techniques, such as deductive verification, model checking, or static analysis, to establish correctness guarantees. Hardware executes machine code produced by compilers, which translate source programs through intermediate representations down to assembly. Consequently, source-level correctness guarantees remain meaningful only if compilation strictly preserves program semantics. Unverified compilation passes risk introducing bugs or invalidating source-level formal proofs.

Initial efforts in verified compilation established foundational methodologies for proving semantic preservation. The CompCert compiler [leroy-09-compcert] relies on simulation diagrams, establishing a correspondence between execution steps of the source and target programs while maintaining a state invariant. CakeML [tan-19-cakeml], a verified compiler for functional languages, combines simulation diagrams with equational reasoning to prove its correctness. Beyond general-purpose compilers, other verified compiler projects focus on specialized application domains. For instance, Jinja [klein-06-jinja] targets object-oriented programs compiled to the Java Virtual Machine, whereas Jasmin [almeida-17-jasmin] verifies a dedicated compiler for high-performance cryptographic software.

Existing verification paradigms exhibit limitations. On the one hand, equational reasoning requires programs to be sufficiently similar. When two programs are written in different languages, use distinct memory representations, or follow different calling conventions, equality may no longer be applicable to state the soundness of the transformation. One must instead establish that target behaviors safely refine source behaviors. On the other hand, traditional simulation diagrams grow complex because they require constructing a global invariant upfront. This invariant must relate entire machine configurations and whole memory states across all program points.

Recent developments in formal methods provide techniques to address these issues. Modern coinductive simulation frameworks, such as the *Stuttering for Free* framework by cho-23-stuttering or *Interaction Trees* [xia-20-itrees], simplify reasoning about program pairs whose execution steps do not correspond one-to-one. In parallel, *relational separation logics* enable local reasoning across dual executions. They allow memory resources to be tracked, partitioned, and treated independently between source and target heaps, avoiding global memory injections. For instance, Simuliris [gaher-22-simuliris] uses a relational separation logic to verify source-to-source program transformations, and allain-25-tmc employ relational separation logic to validate the Tail Modulo Cons optimization.

Research Problem

In verified compilation, semantic preservation is ultimately characterized by a backward simulation. This ensures that every observable behavior of the target program is permitted by the source program. However, existing compilers such as CompCert and CakeML primarily establish forward simulations. Forward simulations are simpler to construct, but they only imply backward simulations when the target language is deterministic. Supporting target non-determinism, such as non-deterministic assignments or concurrency, requires establishing a backward simulation directly.

Furthermore, existing verified compilers rely on global memory invariants and whole-program simulation relations. Using a relational separation logic offers a promising alternative: it allows transformations to be verified by reasoning locally about modified heap cells without considering the entire memory.

This internship investigates whether recent developments in simulation relations and separation logic can be integrated into a relational separation logic framework. To our knowledge, no existing verified compiler incorporates a relational separation logic to prove the soundness of its optimization passes. In short, this approach relaxes constraints present in existing systems, such as the determinism requirements imposed by CompCert and CakeML.

Contributions

We developed a mechanized logic framework addressing these challenges. At its core, we use a coinductive safe backward simulation relation. This frees our approach from determinism restrictions on the source and target languages. The simulation relation is asymmetric: it permits target executions to exploit undefined behaviors in the source, enabling optimizations that assume source execution does not get stuck.

Building upon this simulation relation, we defined a relational separation logic that enables local reasoning over fragments of source and target heaps. The logic is parameterized by the operational semantics of the source and target languages under comparison. Our framework uses this logic to specify and verify relational judgments between program components.

We instantiated this framework with a low-level imperative language, μ RTL, based on the Register Transfer Language used in CompCert. We formalized the operational semantics for this μ RTL dialect. It features constructs for arithmetic operations, dynamic heap allocation and deallocation, and function calls.

Finally, we established a system of relational Hoare rules for μ RTL programs. This proof system comprises two distinct categories of rules. Asynchronous rules govern unilateral execution steps taken by either the source or the target program. These rules enable target-side optimizations by exploiting the absence of source undefined behavior. Synchronous rules govern joint execution steps when both programs advance simultaneously on matching instructions, such as synchronized function calls.

Arguments Supporting Its Validity

We evaluated our framework by verifying the correctness of two concrete program transformations and two general compiler optimizations. For the concrete transformations, one exploits the absence of undefined behavior in the source to eliminate a redundant zero-check after a division, while the other hoists an invariant memory read out of a loop body. For the general optimizations, the first is a branch tunneling pass that eliminates redundant jumps across arbitrary control-flow graphs. The second, which is omitted from the detailed exposition of this report, optimizes redundant instructions, such as superfluous register copies.

Beyond these case studies, we established metatheoretic properties confirming framework soundness. We proved that our relational separation logic soundly implies the underlying backward simulation relation, ensuring semantic preservation. Additionally, we proved that our simulation formulation, which is easier to work with, is equivalent to standard backward simulation definitions used in the literature [leroy-09-compcert, gaher-22-simuliris, allain-25-tmc].

The entire framework has been mechanized in the Rocq proof assistant [rocq]. The development relies on the coinduction Rocq library [rocq-coinduction] and the stdpp library [rocq-stdpp]. Furthermore, our logic is compatible with the Iris proof mode [krebbbers-18-mosel].

Future Work

This work provides a first step for several research perspectives. While current case studies focus on control-flow simplifications and local rewrites, future work will target data-flow optimizations. We plan to verify data-flow transformations ranging from constant propagation to more complex optimizations such as selective defunctionalization.

A second direction involves multi-language verification. Although our logic is parameterized to relate distinct languages, our current case studies operate within a single intermediate representation. Future work will apply the framework to translation passes between different intermediate languages.

Finally, we aim to support richer language features. While our logic framework accommodates non-determinism, our current μ RTL semantics features only limited non-determinism from memory allocation. We plan to investi-

⁹⁰ gate languages featuring concurrency, structured control, first-class closures, or effect handlers. In the long term,
⁹¹ these techniques could be applied to CompCert, notably to support concurrency, and pave the way toward a
⁹² verified OCaml compiler.

1 Verified Compilation Background

1.1 Small-Step Operational Semantics

Giving a formal semantics to a programming language provides a rigorous mathematical foundation for its execution. Small-step operational semantics models computation as a sequence of elementary transitions between program configurations. Rather than evaluating compound expressions in a single step, it describes how internal configurations evolve under primitive operations. This style directly captures non-termination as infinite transition sequences and permits the inspection of intermediate execution states.

Formally, a small-step operational semantics is defined by a transition system over a set of configurations C , equipped with a binary reduction relation $\rightarrow \subseteq C \times C$. In our framework, a configuration is represented as a pair $(s, m) \in S \times M$, where $s \in S$ denotes an internal program state and $m \in M$ denotes a memory state. The structure of the internal state s depends on the language under study: for instance, s may be a term in a pure λ -calculus with substitution semantics; a term paired with an environment in an environment-based semantics; a program counter and an environment; or, as in the μ RTL language introduced later in this report, a call stack, a currently active function, a program counter, and a register bank.

The transition relation $(s, m) \rightarrow (s', m')$ indicates that configuration (s, m) reduces in a single step to (s', m') . The global program code and declarations are implicitly fixed within the transition relation. We denote by $\rightarrow^*$ the reflexive transitive closure of $\rightarrow$. The transition system need not be deterministic: a configuration may admit several outgoing transitions.

To characterize execution endpoints, the semantics is parameterized by a set of observable values val and a partial function $\text{is_final} : S \rightarrow \text{option}(\text{val})$. A configuration (s, m) is *final* when $\text{is_final}((s)) = \text{Some}(v)$ for some return value $v \in \text{val}$. For such final configurations, no further reduction steps are possible: for every memory m and value v , if $\text{is_final}((s)) = \text{Some}(v)$, there exists no configuration (s', m') such that $(s, m) \rightarrow (s', m')$.

We say that a configuration $c = (s, m)$ can make progress, written $\text{can_progress}() c$, if there exists a configuration c' such that $c \rightarrow c'$. Conversely, a configuration c is *stuck*, written $\text{stuck}() c$, if it is not a final configuration ($\text{is_final}((s)) = \text{None}$) and cannot perform any transition ($\neg \text{can_progress}() c$). Stuck configurations represent execution failures where the operational semantics is undefined. In an imperative setting, a configuration becomes stuck when dereferencing an unallocated pointer, dividing by zero, or branching to an invalid control-flow node.

1.2 Program Behaviors

Equipped with a small-step operational semantics, we can formally classify the observable behaviors of a program execution. Because the transition system may be non-deterministic, an execution path starting from an initial configuration $c_0 = (s_0, m_0)$ can yield multiple outcomes. The syntax of observable behaviors b is defined by the following grammar:

$$b ::= \text{Terminating}(v, m) \mid \text{Diverging} \mid \text{Undefined}$$

where $v \in \text{val}$ and $m \in M$. For a given initial configuration c_0 , an execution path exhibits one of three observable behaviors:

- c_0 *terminates* with value v and memory m , written $c_0 \Downarrow \text{Terminating}(v, m)$, if execution reaches a final configuration (s', m) such that $c_0 \rightarrow^* (s', m)$ and $\text{is_final}((s')) = \text{Some}(v)$.
- c_0 *diverges*, written $c_0 \Downarrow \text{Diverging}$, if there exists an infinite sequence of transitions starting from c_0 . This encompasses all non-terminating executions, without requiring that every execution path diverges.
- c_0 exhibits *undefined behavior*, written $c_0 \Downarrow \text{Undefined}$, if execution reaches a stuck configuration: there exists c' such that $c_0 \rightarrow^* c'$ and $\text{stuck}() c'$.

We write $b \in c_0$ (or $c_0 \Downarrow b$) to denote that behavior b is exhibited by configuration c_0 . A complete program P comes equipped with an initial configuration c_{init} . We extend the definition of behaviors to programs by setting:

$$b \in P := b \in c_{\text{init}}$$

Every configuration admits at least one observable outcome:

Lemma 1. For any configuration c , there exists a behavior b such that $b \in c$.

1.3 Program Refinement

The objective of a verified compiler is to guarantee semantic preservation: the compiled target executable must faithfully preserve the behaviors of the source program. In verified compilation, semantic preservation is formalized as a *behavioral refinement* between the compiled target program T and the source program S .

A direct formulation is *rigid behavioral refinement*, requiring every behavior of the target program to be an admitted behavior of the source program:

$$T \sqsubseteq S := \forall b, b \in T \implies b \in S \quad (1)$$

Rigid refinement ensures that compilation never introduces spurious behaviors, while permitting the compiler to resolve non-deterministic choices present in the source.

However, rigid behavioral refinement (??) is overly restrictive for optimizing compilers. First, the target and source languages may manipulate distinct value representations or memory layouts, so terminal values and memories should be related by an observable equivalence rather than strict identity. Second, when a source program encounters undefined behavior, the semantics leaves execution undefined. An optimizing compiler is then permitted to eliminate erroneous branches, transform invalid operations into benign code, or exhibit any convenient target behavior. Third, if a source execution diverges, the compiled program must diverge as well.

To account for these optimization principles, compiler correctness relies on a *flexible behavioral refinement* (also termed skewed refinement). We first equip behaviors with a preorder $\leq_{\text{beh}}$, parameterized by relations on values ($\sim_v$) and memories ($\sim_m$):

$$\begin{array}{c} \frac{v_t \sim_v v_s \quad m_t \sim_m m_s}{\text{Terminating}(v_t, m_t) \leq_{\text{beh}} \text{Terminating}(v_s, m_s)} \text{BEH-TERM} \qquad \frac{}{\text{Diverging} \leq_{\text{beh}} \text{Diverging}} \text{BEH-DIV} \\[10pt] \frac{}{b_t \leq_{\text{beh}} \text{Undefined}} \text{BEH-UNDEF} \end{array}$$

The rules of this preorder admit an intuitive reading:

- Producing a value v_t and memory m_t in the compiled program is justified by the possibility of producing a corresponding value v_s and memory m_s in the source program.
- Divergence in the compiled program is justified by the possibility of divergence in the source program.
- Any behavior b_t in the compiled program is justified by the possibility of undefined behavior in the source program.

We then define *flexible behavioral refinement* between a target program T and a source program S :

$$T \leq S := \forall b_t, b_t \in T \implies \exists b_s, b_s \in S \wedge b_t \leq_{\text{beh}} b_s \quad (2)$$

This relation is similarly defined between configurations $t \leq s$. Whenever $T \leq S$ holds, every observable behavior produced by the compiled program is validated by a matching execution in the source program, up to undefined behavior exploitation.

1.4 Simulation Relations

While the refinement relation $t \leq s$ is formulated globally over whole-program behaviors, proving it directly over full execution traces is impractical. Compilers perform transformations whose correctness is best verified step by step. To bridge this gap, verified compilation relies on *simulation relations*, which maintain an invariant relating intermediate configurations of the source and target executions. Simulation relations are established in two flavors:

Backward simulation: each execution step taken by the target program must be matched by corresponding steps in the source program. Because it maps every target execution path back to an admissible source path, a backward simulation directly establishes behavioral refinement $t \leq s$.

Forward simulation: each execution step taken by the source program is matched by steps in the target program. Forward simulations are often easier to establish, as compiler passes transform source constructs

into target code. However, a forward simulation implies a backward simulation only when the target language is deterministic. For languages with non-deterministic constructs, one must establish a backward simulation directly.

1.5 Coinductive Predicates

Proving a simulation relation requires relating operational transitions of the target and source programs. When executions terminate, they consist of finite reduction sequences ending in final configurations. Such finite behaviors are formalized using *inductive* predicates. However, programs designed to run indefinitely produce infinite reduction sequences that never encounter a base case. Establishing a simulation inductively for diverging executions is impossible, as it would require an infinite derivation tree, which inductive definitions forbid. To reason about infinite execution traces, we rely on *coinductive* predicates.

The presentation adopted below, which interprets coinduction in terms of infinite derivation trees, is an intuitive presentation adapted to our setting. For foundational treatments of coinduction, we refer the reader to **milner-89-communicationgimenez-98-tutorialpous-16-cawu**. Throughout this report, we specify inductive and coinductive relations through inference rules:

- *Inductive rules* are denoted with a single horizontal bar:

$$\frac{H_1 \quad \dots \quad H_n}{\mathcal{I}}$$

A judgment holds inductively if and only if it can be established through a *finite derivation tree*, grounded in base cases.

- *Coinductive rules* are denoted with a double horizontal bar:

$$\frac{H_1 \quad \dots \quad H_n}{\mathcal{C}}$$

They generalize inductive rules to judgments admitting *potentially infinite derivation trees*.

For example, finite lists are defined inductively, requiring every list to terminate at nil. In contrast, infinite streams are defined coinductively, allowing endless constructor applications without base cases:

$$\frac{}{\text{nil} \in \text{List}(\mathbb{N})} \quad \frac{n \in \mathbb{N} \quad l \in \text{List}(\mathbb{N})}{\text{cons}(n, l) \in \text{List}(\mathbb{N})} \quad \frac{n \in \mathbb{N} \quad s \in \text{Stream}(\mathbb{N})}{\text{scons}(n, s) \in \text{Stream}(\mathbb{N})}$$

These distinct derivation structures govern dual proof principles:

- One reasons by *induction* when the *hypothesis* is an inductive judgment. The proof proceeds by inversion or case analysis on the constructors of the finite derivation. The induction hypothesis may only be applied to *strictly smaller subterms*, guaranteeing finite descent toward a base case.
- One reasons by *coinduction* when the *goal* to be established is a coinductive judgment. The proof constructs a potentially infinite derivation tree. The coinductive hypothesis refers to the goal itself, but it may only be invoked *underneath a constructor* (scons for streams). This requirement, known as *productivity* or *guardedness*, ensures that the corecursive definition is well-founded.

A detailed example illustrating coinductive reasoning is provided in ???. The Rocq theorem prover [**rocq**] supports coinductive predicates natively. In our formalization, we use the `coinduction` library developed by **rocq-coinduction** to facilitate reasoning with coinductive simulation invariants.

1.6 Simulation Schemes & Stuttering

Equipped with coinductive predicates, we can define simulation relations between intermediate configurations. In general, the source and target languages may differ: target configurations $t = (s_t, m_t)$ evolve under $\rightarrow_t$ with finality test `is_final()` t , while source configurations $s = (s_s, m_s)$ evolve under $\rightarrow_s$ with finality test `is_final()` s . A backward simulation relates a target configuration t to a source configuration s , ensuring that every execution step of the target is matched by the source.

213 For terminating executions, the simulation requires that final configurations yield related values and memories:

$$\frac{\text{is_final}() t(s_t) = \text{Some}(v_t) \quad \text{is_final}() s(s_s) = \text{Some}(v_s) \quad v_t \sim_v v_s \quad m_t \sim_m m_s}{t \sim_l s} \quad (3)$$

214 When the source program encounters undefined behavior (stuck (s) s), flexible refinement permits the target to
215 exhibit arbitrary behavior. The simulation is therefore immediately validated without requiring progress from
216 the target:

$$\frac{\text{stuck}(s) s}{t \sim_l s} \quad (4)$$

217 For non-final configurations where the source is not stuck, the relation requires that the target can make progress,
218 and that every transition it performs is matched by a step in the source. This step condition is expressed
219 coinductively:

$$\frac{\text{can_progress}(t) t \quad \forall t', t \twoheadrightarrow_t t' \implies \exists s', s \twoheadrightarrow_s s' \wedge t' \sim_l s'}{t \sim_l s} \quad (5)$$

220 The relation $\sim_l$ defined by ????? is a *lock-step* backward simulation: the target and source programs advance at
221 the exact same pace. Under non-determinism, the target and source configurations are treated asymmetrically:

- 222 • The target program exhibits *demonic* non-determinism: the simulation must account for all reachable
223 successor configurations t' .
- 224 • The source program exhibits *angelic* non-determinism: the proof only needs to exhibit the existence of a
225 single matching successor s' .

226 This asymmetry matches the quantification structure of refinement: we must account for every non-deterministic
227 path chosen by the target and justify it with an admissible path in the source.

228 While lock-step simulations suffice for simple transformations that preserve step counts, realistic compiler passes
229 break this one-to-one correspondence. An optimization pass such as branch tunneling removes redundant jump
230 instructions, causing the source program to execute steps that have no counterpart in the target. Conversely, low-
231 ering transformations decompose a single high-level source instruction into multiple low-level target instructions,
232 requiring the target to execute several steps before the source advances.

233 To accommodate passes that alter step counts, one must allow *asynchronous* execution steps, where one program
234 takes a step while the other remains stationary. Such a transition is called a *stuttering* step. A naive coinductive
235 formulation might introduce unrestricted stuttering rules:

$$\frac{\text{can_progress}(t) t \quad \forall t', t \twoheadrightarrow_t t' \implies t' \sim s}{t \sim s} \quad \frac{s \twoheadrightarrow_s s' \quad t \sim s'}{t \sim s}$$

236 However, adding these unconstrained rules invalidates the fundamental soundness property that *simulation*
237 *implies behavioral refinement* ($t \sim s \implies t \leq s$). The source of unsoundness lies in target stuttering: without
238 restriction, a target program could perform an infinite sequence of stuttering steps without the source program
239 ever stepping. Under such a flawed definition, a diverging target program would be related to any source
240 configuration, including terminating or stuck source programs. In contrast, infinite source stuttering alone
241 does not compromise refinement: if the source steps infinitely while the target remains stationary, the source
242 execution diverges ($s \Downarrow \text{Diverging}$), which validates any target behavior under $\leq_{\text{beh}}$. Nonetheless, uncontrolled
243 stuttering would obscure termination arguments.

244 To ensure soundness, target stuttering must be strictly finite. This is achieved by equipping the simulation with
245 a well-founded measure $(W, \prec)$. The relation is indexed by a measure $i \in W$, written $t \sim_e^i s$. Whenever a
246 stuttering step occurs, the index strictly decreases according to $\prec$. When both programs step simultaneously,
247 progress is established and the measure may be freely reset.

248 Because stuttering is bounded by an explicit measure, this relation is called the *explicit simulation* [cho-23-stuttering].
249 The inference rules defining it are presented in ??.

250 The rules ESourceSteps and ETargetSteps are *inductive* (denoted with a single horizontal bar). Because $(W, \prec)$
251 is well-founded, any chain of consecutive stuttering transitions must be finite, preventing infinite stuttering cycles.

$$\begin{array}{c}
\frac{\text{is_final}() t(s_t) = \text{Some}(v_t) \quad \text{is_final}() s(s_s) = \text{Some}(v_s) \quad v_t \sim_v v_s \quad m_t \sim_m m_s}{t \sim_e^i s} \text{ERELATED} \quad \frac{\text{stuck}(s) s}{t \sim_e^i s} \text{ESOURCES} \\
\\
\frac{s \rightarrow_s s' \quad \exists i', i' \prec i \wedge t \sim_e^{i'} s'}{t \sim_e^i s} \text{ESOURCESTEPS} \\
\\
\frac{\text{can_progress}(t) t \quad \forall t', t \rightarrow_t t' \implies \exists i', i' \prec i \wedge t' \sim_e^{i'} s}{t \sim_e^i s} \text{ETARGETSTEPS} \\
\\
\frac{\text{can_progress}(t) t \quad \forall t', t \rightarrow_t t' \implies \exists s', \exists i', s \rightarrow_s s' \wedge t' \sim_e^{i'} s'}{t \sim_e^i s} \text{EBOTHSTEPS}
\end{array}$$

Figure 1: Derivation rules of the *explicit simulation*.

The rule EBOTHSTEPS is the sole *coinductive* constructor (denoted with a double horizontal bar). It generates infinite derivation trees to justify non-terminating executions. However, EBOTHSTEPS requires synchronizing target and source steps at each mutual progress point. In practice, synchronizing progress steps and constructing global stuttering measures imposes a heavy proof burden for complex compiler transformations.

2 The Free Simulation

While explicit simulations effectively prevent infinite stuttering, they impose a substantial proof burden. Every stuttering transition requires exhibiting a strictly decreasing measure in a well-founded order. Constructing such a measure can be difficult in practice, particularly when compiler optimizations rearrange loop structures. In such scenarios, finding a global metric that strictly decreases on every asynchronous step while remaining invariant across complex control transfers is often impractical.

To overcome this rigidity, we adapt the *Stuttering for Free* framework introduced by **cho-23-stuttering**. We extend their foundational technique to accommodate skewed simulation relations. We have formalized and verified all underlying theoretical results within our Roc development.

2.1 Skewed Implicit Simulation

A possible approach to eliminating explicit stuttering measures is to integrate inductive and coinductive reasoning into a single relation. In this formulation, finite sequences of asynchronous execution steps are justified inductively, whereas mutual progress steps are justified coinductively. This family of relations is called *implicit simulations*. We define a skewed implicit simulation relation, denoted $t \lesssim s \{\Phi\}$, whose inference rules are presented in ??.

To combine inductive and coinductive reasoning within the same definition, the relation $t \lesssim s \{\Phi\}$ is defined as the greatest fixed point¹ of an inductive operator. The rules IRELATED, ISOURCESTUCK, ISOURCESTEPS, and ITARGETSTEPS serve as inductive constructors, while the rule IBOTHSTEPS acts as the coinductive constructor.

The rule IRELATED characterizes terminating executions. Instead of demanding syntactic equality between terminal values, it is parameterized by a postcondition Φ that relates the final values and the concluding memory states of the target and source programs. The predicate `both_final` is defined as follows:

$$\begin{aligned}
\Phi : \text{val}_t \rightarrow \text{val}_s \rightarrow \text{mem} \rightarrow \text{mem} \rightarrow \text{Prop} \\
\text{both_final}_\Phi((t, m_t), (s, m_s)) := \exists v_t v_s, t = \text{Some}(v_t) \wedge s = \text{Some}(v_s) \wedge \\
\Phi(v_t, v_s, m_t, m_s)
\end{aligned}$$

The assertion `both_finalΦ(t, s)` holds when both target and source configurations have reached final states, producing values v_t, v_s and memories m_t, m_s that satisfy Φ . This parameterization enables the simulation to enforce relational invariants on memories at the end of execution.

¹The relationship between greatest fixed points and coinduction follows from the Knaster–Tarski theorem; see [pous-16-cawu, schaffer-17-tower] for comprehensive treatments.

$$\begin{array}{c}
\frac{\text{both_final}_\Phi(t, s)}{t \lesssim s \{\Phi\}} \text{ IRELATED} \qquad \frac{\text{stuck}(P_s) s}{t \lesssim s \{\Phi\}} \text{ ISOURCESTUCK} \\
\\
\frac{P_s \vdash s \rightarrow s' \quad t \lesssim s' \{\Phi\}}{t \lesssim s \{\Phi\}} \text{ ISOURCESTEPS} \\
\\
\frac{\text{can_progress}(P_t) t \quad \forall t', P_t \vdash t \rightarrow t' \implies t' \lesssim s \{\Phi\}}{t \lesssim s \{\Phi\}} \text{ ITARGETSTEPS} \\
\\
\frac{\text{can_progress}(P_t) t \quad \forall t', P_t \vdash t \rightarrow t' \implies \exists s', P_s \vdash s \rightarrow s' \wedge t' \lesssim s' \{\Phi\}}{t \lesssim s \{\Phi\}} \text{ IBOTHSTEPS}
\end{array}$$

Figure 2: Derivation rules of the *skewed implicit simulation*.

279 The rule ISOURCESTUCK introduces more asymmetry to the relation. When the source program encounters
 280 undefined behavior, it enters a stuck state in a non-final configuration. In this case, ISOURCESTUCK immediately
 281 validates the simulation, leaving the target program free to perform arbitrary transitions or diverge. This
 282 asymmetry allows compiler transformations that rely on the absence of undefined behavior in well-formed source
 283 programs to eliminate redundant runtime checks.

284 Stuttering is soundly handled without an explicit decreasing measure. Because ISOURCESTEPS and ITARGETSTEPS
 285 are inductive constructors, they permit only finitely many consecutive asynchronous transitions. The coinductive
 286 constructor IBOTHSTEPS requires both programs to execute a synchronized step, to remove the inductive proof
 287 obligation and enable unbounded mutual progress.

288 2.2 Step-Indexed Free Simulation

289 While the implicit simulation eliminates manual stuttering measures, it remains restrictive: the coinductive rule
 290 IBOTHSTEPS demands that both programs step at the exact moment when coinductive reasoning is resumed.
 291 Removing this lock-step synchronization requirement leads to smaller invariants and more direct verification
 292 proofs [cho-23-stuttering]. This decoupling is the core idea developed in the *Stuttering for Free* framework.

293 The *free simulation*, denoted $t \lesssim_i s \{\Phi\}$, overcomes this limitation by decoupling synchronous execution steps
 294 from coinductive progress. The relation is parameterized by two well-founded index domains, $(J, \prec_J)$ for the
 295 target side and $(I, \prec_I)$ for the source side. These domains track the progress accumulated by each side since the
 296 previous coinductive step. The formal inference rules of the skewed free simulation are presented in ???. Like the
 297 implicit simulation, it accommodates arbitrary postconditions Φ and uses the rule FSOURCESTUCK to exploit any
 298 undefined behavior in the source.

$$\begin{array}{c}
\frac{\text{both_final}_\Phi(t, s)}{t \lesssim_i s \{\Phi\}} \text{ FRELATED} \qquad \frac{\text{stuck}(P_s) s}{t \lesssim_i s \{\Phi\}} \text{ FSOURCESTUCK} \\
\\
\frac{P_s \vdash s \rightarrow s' \quad t \lesssim_{i'} s' \{\Phi\}}{t \lesssim_i s \{\Phi\}} \text{ FSOURCESTEPS} \\
\\
\frac{\text{can_progress}(P_t) t \quad \forall t', P_t \vdash t \rightarrow t' \implies \exists j', t' \lesssim_{j'} s \{\Phi\}}{t \lesssim_i s \{\Phi\}} \text{ FTARGETSTEPS} \\
\\
\frac{j' \prec_J j \quad i' \prec_I i \quad t \lesssim_{i'} s \{\Phi\}}{t \lesssim_i s \{\Phi\}} \text{ FPROGRESS}
\end{array}$$

Figure 3: Derivation rules of the *skewed free simulation*.

299 The design of the free simulation relies on a fundamental shift in perspective regarding indices. In an explicit
 300 simulation, an index represents an upper bound on *future* stuttering steps. In the free simulation, indices record
 301 *past* progress accumulated since the previous coinductive step.

The source and target programs take transitions asynchronously via the inductive rules FTARGETSTEPS and FSOURCESTEPS. Whenever a program takes a step, its corresponding step index is updated (j' or i'). We note that j' and i' are not required to be immediate successors of j and i . Instead, the user is free to select any new index, smaller or larger.

This flexibility does not compromise soundness. Indeed, the only rule that enables coinductive reasoning is FPROGRESS. This rule requires both indices to strictly decrease, from j down to $j' \prec_J j$ and from i down to $i' \prec_I i$. Because the relations $\prec_J$ and $\prec_I$ are well-founded, establishing a simulation between non-terminating executions demands that FPROGRESS be applied infinitely often, which in turn requires indices to be repeatedly increased through FTARGETSTEPS and FSOURCESTEPS. This structure decouples stepping from coinductive synchronization, allowing multiple asynchronous transitions on either side to be matched flexibly.

Moreover, performing coinductive reasoning over the free simulation is equivalent to the following principle: to prove $t \prec_{j \prec i} s \{\Phi\}$, one may assume the coinductive hypothesis:

$$\forall j' \succ_J j, \forall i' \succ_I i, \quad t \prec_{j' \prec i'} s \{\Phi\}.$$

Intuitively, this principle allows one to establish that two programs simulate each other by assuming that the simulation relation already holds for the current states if they are reached after progress on both sides. A concrete example illustrating step-indexed reasoning in the free simulation is presented in ??.

2.3 Properties

Proving a free simulation for smaller indices implies that it holds for larger indices. This monotonicity property is valid not only as an external lemma, but also within coinductive proofs. This property is captured by index monotonicity:

Lemma 2 (Monotonicity of Free Simulation). If $t \prec_{j \prec i} s \{\Phi\}$ holds, then for all indices $j \prec_J j'$ and $i \prec_I i'$, and any weaker postcondition $\Phi \implies \Phi'$, the simulation $t \prec_{j' \prec i'} s \{\Phi'\}$ holds.

Next, the free simulation is sound with respect to the skewed behavioral refinement:

Theorem 1 (Soundness of Free Simulation). Let P_t and P_s be programs. Let t be the initial state of P_t and s be the initial state of P_s . If $t \prec_{j \prec i} s \{=\}$ holds for some indices j and i , then $P_t \leq P_s$.

In addition to soundness, we established the formal equivalence among the three simulation frameworks discussed: the free simulation (defined in ??), the implicit simulation (defined in ??), and the explicit simulation (defined in ??). This equivalence is established by chaining cyclic implications:

$$\text{Explicit Simulation} \xRightarrow{\textcircled{1}} \text{Implicit Simulation} \xRightarrow{\textcircled{2}} \text{Free Simulation} \xRightarrow{\textcircled{3}} \text{Explicit Simulation}$$

The implications ① and ② proceed directly by simulation coinduction. The last implication ③ is the most intricate: it requires synthesizing a well-founded stuttering measure from the derivation tree of the free simulation. A direct consequence of this equivalence is the index irrelevance theorem:

Theorem 2 (Index Irrelevance). Let $(J, \prec_J)$ and $(I, \prec_I)$ be well-founded relations without isolated elements². If there exist indices $j \in J$ and $i \in I$ such that $t \prec_{j \prec i} s \{\Phi\}$, then for any other well-founded structures $(J', \prec_{J'})$ and $(I', \prec_{I'})$, and any indices $j' \in J'$ and $i' \in I'$, the relation $t \prec_{j' \prec i'} s \{\Phi\}$ holds.

Our statement refines the index irrelevance theorem formulated by **cho-23-stuttering**. Their original formulation, phrased in terms of ordinals, implicitly assumes that every element participates in the order. However, isolated elements could cause FPROGRESS transitions to be uninhabited, preventing the equivalence. This distinction was identified during our Rocq formalization of the metatheory of the free simulation.

This theorem implies that any free simulation derivation can be carried out using the two-element boolean order $\mathbb{B} = \{\text{false} \prec \text{true}\}$:

$$(\exists j \in J, i \in I, t \prec_{j \prec i} s \{\Phi\}) \iff (\exists j, i \in \mathbb{B}, t \prec_{j \prec i} s \{\Phi\}).$$

Under the boolean order, false indicates that a program has not yet taken a step, whereas true signals that progress has occurred. This means that, in theory, the free simulation proof does not strictly require tracking the

²An element is isolated if it has no predecessors or successors in the relation.

exact step counts of individual programs, but only whether progress has occurred since the last coinductive step. Nevertheless, in the remainder of our work, we adopt the natural numbers $(\mathbb{N}, <)$ as our index domain. Indeed, natural number indices correspond directly to the number of steps executed during symbolic evaluation, and they provide some intuition that facilitates the interactive verification in Rocq.

3 The Register Transfer Language (μ RTL)

To evaluate our framework on realistic compiler intermediate representations, we formalize a low-level imperative language, μ RTL, based on the Register Transfer Language (RTL) of CompCert [leroy-09-compcert]. This language has two kinds of storage: local register banks attached to execution states, which act as local variables, and a global shared heap memory.

3.1 Values, Registers, and Memory

Computations in μ RTL manipulate first-order values. There are four kinds of values, where loc represents allocated heap locations:

$$v \in \text{val} ::= \text{VInt}(z) \mid \text{VBool}(b) \mid \text{VPtr}(l) \mid \text{VUndef} \\ \text{with } z \in \mathbb{Z}, b \in \{\text{true}, \text{false}\}, \text{ and } l \in \text{loc}$$

VUndef represents an undefined value, used to model uninitialized data. It can be copied between registers, stored into memory cells, or passed as function arguments. However, attempting to execute an instruction that requires a specific kind of value, such as an arithmetic calculation, will cause the execution to get stuck.

To relate source and target values, we define a similarity predicate $v_t \bowtie_I v_s$. This predicate is parameterized by a binary relation on locations, $I \subseteq \text{loc} \times \text{loc}$. This relation will be made precise later in ???. The deduction rules of the value similarity predicate are given in ???. We note that the rule VAL-UNDEF is asymmetric: whenever the source program produces an undefined value VUndef , it can be related to any value v_t produced by the target program. This asymmetry allows the compiler to replace undefined source computations with concrete target operations.

$$\begin{array}{c} \frac{z_t = z_s}{\text{VInt}(z_t) \bowtie_I \text{VInt}(z_s)} \quad \text{VAL-INT} \qquad \frac{b_t = b_s}{\text{VBool}(b_t) \bowtie_I \text{VBool}(b_s)} \quad \text{VAL-BOOL} \\[1em] \frac{(l_t, l_s) \in I}{\text{VPtr}(l_t) \bowtie_I \text{VPtr}(l_s)} \quad \text{VAL-PTR} \qquad \frac{}{v_t \bowtie_I \text{VUndef}} \quad \text{VAL-UNDEF} \end{array}$$

Figure 4: The value similarity predicate.

In μ RTL, registers store values that can be actively manipulated in computations. They are referenced by an identifier $r \in \text{reg}$. We do not restrict the number of registers. A μ RTL program can also interact with the heap. The heap is indexed by locations $l \in \text{loc}$, and the program can read, write, allocate, and free memory cells associated with locations.

3.2 Syntax and Program Structure

High-level imperative languages employ *structured* control-flow constructs, such as nested conditionals, while loops, and compound statements. In structured languages, execution paths follow the hierarchical source code structure of the program. In contrast, μ RTL is an *unstructured* language. Control flow in μ RTL is represented as an explicit directed graph whose nodes $pc \in \text{node}$ represent program points. Each instruction explicitly specifies the program points of its successors, eliminating the need for loop or block constructs. The instruction set and operator grammar of μ RTL are defined in ???. A μ RTL function f is a record containing:

$$f \in \text{rtl_function} ::= \left\{ \begin{array}{ll} \text{fn_name} \in \text{ident} & \text{function identifier} \\ \text{fn_regs} \in \text{list reg} & \text{parameter list} \\ \text{fn_entrypoint} \in \text{node} & \text{function entry point} \\ \text{fn_code} \in \text{node} \rightarrow \text{instr} & \text{control-flow graph} \end{array} \right\}$$

375 We enforce the invariant that input parameter registers contain no duplicates. The code map fn_code is a finite
 376 map associating program points with instructions. In the following, we write $f[pc] = \langle i \rangle$ to assert that the code
 377 map of f contains instruction i at program point pc . A μRTL program P consists of a list of functions with a
 378 distinguished entry function. We require that function names in the program are unique, and that the entry
 379 function is present.

$i \in \text{instr}$	$::=$	$\text{nop} \rightarrow pc$	unconditional jump
		$dst := \text{op } \vec{r} \rightarrow pc$	pure arithmetic or logical operation
		$dst := !src \rightarrow pc$	heap memory load
		$!addr := src \rightarrow pc$	heap memory store
		$dst := \text{alloc}() \rightarrow pc$	heap cell allocation
		$\text{free } addr \rightarrow pc$	heap cell deallocation
		$\text{if } cond \text{ then } pc_{\text{true}} \text{ else } pc_{\text{false}}$	conditional branching
		$dst := \text{call } fn(\vec{r}) \rightarrow pc$	function invocation
		$\text{ret } src$	function return
op	$::=$	$\text{Add} \mid \text{Sub} \mid \text{Mul} \mid \text{Div}$	integer arithmetic
		$\text{Incr} \mid \text{Decr}$	increment and decrement
		$\text{ImmInt}(z) \mid \text{ImmBool}(b)$	constant immediate values
		$\text{Move} \mid \text{EqZ}$	register copy and zero test

where $dst, src, addr, cond \in \text{reg}$, $\vec{r} \in \text{list reg}$, $fn \in \text{ident}$, and $pc, pc_{\text{true}}, pc_{\text{false}} \in \text{node}$.

Figure 5: The syntax of μRTL instructions and operators.

3.3 Small-Step Operational Semantics

381 The execution of a μRTL program is formalized as a small-step operational transition system. The configurations
 382 are tuples (σ, s, m) , where σ is a call stack, $m \in \text{memory}$ is the global heap, and $s \in \text{pcstate}$ is a control state
 383 which can take three distinct forms:

- 384 • $\text{State}(f, pc, \rho)$ is the main state. It refers to an execution occurring inside function f at program point pc ,
 385 with local register bank ρ .
- 386 • $\text{CallState}(f, \vec{v})$ represents the beginning of the invocation of function f with argument values $\vec{v}$.
- 387 • $\text{ReturnState}(v)$ represents the completion of a function execution with return value v .

388 The call stack σ is a list of stack frames. Each stack frame $\text{Stackframe}(dst, f, pc, \rho)$ stores the caller context: the
 389 destination register dst designated to receive the return value, the caller function f , the return program point
 390 pc , and the caller register bank ρ . Structuring transitions through explicit $\text{CallState}(f, \vec{v})$ and $\text{ReturnState}(v)$
 391 configurations cleanly decouples caller and callee environments. In CompCert [leroy-09-compcert], this
 392 abstraction is also used to integrate external function calls that jump directly from a call state to a return state.

393 A register bank and the memory heap are formalized as shown in ?? . By explicitly recording Freed cells rather
 394 than removing locations from the domain of m , our memory model distinguishes dangling pointer dereferences
 395 and double frees from accesses to unallocated memory.

396 The step relation $P \vdash s \rightarrow s'$ is defined by inference rules, selected rules of which are presented in ?? . Execution
 397 of a function progresses through three main phases ($\text{CallState} \rightarrow \text{State} \rightarrow \text{ReturnState}$):

- 398 1. In the beginning, the state is a call state $\text{CallState}(f, \vec{v})$. We use the rule STEP-ENTRY to step into the body
 399 of the function at its entry point. This initializes the register bank with the values given for the input
 400 registers of f . The resulting state is an active state $\text{State}(f, pc_0, \rho)$.
- 401 2. Then we use the instruction rules to execute the body of the function. These rules can only apply if they
 402 correspond to the next instruction to be executed, referenced by pc in f . We note that evaluation of some
 403 instructions is partial. For instance, in the STEP-OP rule, if $\text{eval_op}(op, \vec{v})$ returns None , the execution is
 404 stuck.
- 405 3. When the function reaches a return instruction, the rule STEP-RET is applied. It evaluates its return register
 406 and enters a return state $\text{ReturnState}(v)$.

$\rho \in \text{regbank}$	$::=$	$\text{reg} \rightarrow \text{val}$	register bank
$m \in \text{memory}$	$::=$	$\text{loc} \rightarrow \text{cell}$	memory heap
$c \in \text{cell}$	$::=$	$\text{Allocated}(v) \mid \text{Freed}$	heap memory cell

Notation	Operation Semantics
$\rho[r]$	Read register r ; yields VUndef if $r \notin \text{dom}(\rho)$
$\rho[\vec{r}]$	Pointwise extension to register lists, returning $\vec{v}$
$\rho[r \mapsto v]$	Register assignment updating r to value v
$m[l]$	Read cell at location l
$m[l \mapsto \text{Allocated}(v)]$	Update allocated cell at l to value v
$m \uplus \{l \mapsto \text{Allocated}(v)\}$	Allocate fresh cell $l \notin \text{dom}(m)$ initialized with value v
$m[l \mapsto \text{Freed}]$	Deallocate active cell at l , setting its status to Freed

Figure 6: Formalization of register banks and memory heap operations.

4. If the call stack is not empty, the caller resumes execution via **STEP-RETURN**, popping the top stack frame, writing the return value into register dst , and branching to the next pc .
5. When a function call occurs, the rule **STEP-CALL** is used. The caller evaluates the argument registers, pushes a fresh stack frame recording its continuation point pc' and register bank ρ , and transitions to a call state $\text{CallState}(f_{\text{callee}}, \vec{v})$.

An execution state is final when the call stack is empty and the program is in a return state. The final value is the one stored in the control state.

3.4 Stack Modularity and Execution Invariants

Reasoning directly about executions with arbitrary call stacks introduces unnecessary proof complexity. Without stack modularity, one would need to maintain an explicit invariant over caller stack frames and prove that it is preserved across every step. To achieve modularity, we establish lemmas showing that function executions can be analyzed in isolation with an empty call stack $\sigma = []$, and then embedded into arbitrary execution contexts.

When verifying a function call, the callee is verified starting from an empty call stack $([], \text{CallState}(f, \vec{v}), m)$. Once the callee reduces to $([], \text{ReturnState}(v), m')$, the derivation is lifted back into the caller's stack context, reconnecting the callee's execution with the caller's continuation.

4 Relational Separation Logic

Having defined our free simulation framework and instantiated it with μRTL , we require a program logic to specify and verify relational properties of program executions. Reasoning about memory transformations using global heap predicates imposes a heavy verification burden, as every intermediate step must maintain global invariants. To enable local reasoning over pairs of memory heaps, we develop a relational separation logic.

4.1 Separation Logic

Separation logic [**chargueraud-20-sl**] establishes a foundation for modular reasoning about programs that manipulate memory. In standard Hoare logic, assertions describe properties of the entire heap. Establishing that an operation modifies only a restricted portion of memory requires explicitly proving that all untouched heap regions remain invariant, an issue known as the frame problem. Separation logic solves this challenge by introducing the separating conjunction $P * Q$, which asserts that the memory can be split into two disjoint sub-heaps satisfying P and Q , respectively.

In verified compilation, we reason relationally about two program executions: the target program and the source program. Because each execution operates over its own memory heap, we formulate a relational separation logic [**yang-07-rsl**] whose assertions describe pairs of memory states (m_t, m_s) .

$\frac{f[pc] = \langle dst := op \vec{r} \rightarrow pc' \rangle \quad \rho[\vec{r}] = \vec{v} \quad \text{eval_op}(op, \vec{v}) = \text{Some}(v)}{P \vdash (\sigma, \text{State}(f, pc, \rho), m) \rightarrow (\sigma, \text{State}(f, pc', \rho[dst \mapsto v]), m)} \quad \text{STEP-OP}$	
$\frac{f[pc] = \langle dst := \text{alloc}() \rightarrow pc' \rangle \quad l \notin \text{dom}(m) \quad v \in \text{val}}{P \vdash (\sigma, \text{State}(f, pc, \rho), m) \rightarrow (\sigma, \text{State}(f, pc', \rho[dst \mapsto \text{VPtr}(l)]), m \uplus \{l \mapsto \text{Allocated}(v)\})} \quad \text{STEP-ALLOC}$	
$\frac{f[pc] = \langle \text{free addr} \rightarrow pc' \rangle \quad \rho[addr] = \text{VPtr}(l) \quad m[l] = \text{Allocated}(_) }{P \vdash (\sigma, \text{State}(f, pc, \rho), m) \rightarrow (\sigma, \text{State}(f, pc', \rho), m[l \mapsto \text{Freed}])} \quad \text{STEP-FREE}$	
$\frac{f[pc] = \langle dst := \text{call } fn(\vec{r}) \rightarrow pc' \rangle \quad \text{find_fun}(P, fn) = \text{Some}(f_{\text{callee}}) \quad \rho[\vec{r}] = \vec{v}}{P \vdash (\sigma, \text{State}(f, pc, \rho), m) \rightarrow (\text{Stackframe}(dst, f, pc', \rho) :: \sigma, \text{CallState}(f_{\text{callee}}, \vec{v}), m)} \quad \text{STEP-CALL}$	
$\frac{ \vec{v} = \text{fn_regs}(f) \quad \rho = \text{init_regs}(\text{fn_regs}(f), \vec{v})}{P \vdash (\sigma, \text{CallState}(f, \vec{v}), m) \rightarrow (\sigma, \text{State}(f, \text{fn_entrypoint}(f), \rho), m)} \quad \text{STEP-ENTRY}$	
$\frac{f[pc] = \langle \text{ret src} \rangle \quad \rho[src] = v}{P \vdash (\sigma, \text{State}(f, pc, \rho), m) \rightarrow (\sigma, \text{ReturnState}(v), m)} \quad \text{STEP-RET}$	
$P \vdash (\text{Stackframe}(dst, f, pc', \rho) :: \sigma, \text{ReturnState}(v), m) \rightarrow (\sigma, \text{State}(f, pc', \rho[dst \mapsto v]), m) \quad \text{STEP-RETURN}$	

Figure 7: Selected rules of the small-step operational semantics of μRTL .

In modern interactive proof assistants, the Iris framework [jung-18-iris, krebbers-17-essence] serves as the standard platform for separation logic in Rocq. Iris provides expressive mechanisms for higher-order concurrent reasoning, including step-indexed models, higher-order ghost state, and user-defined resource algebras. For our setting, these advanced features are not needed. Instead, we formalize our relational separation logic directly as a shallow embedding in Rocq. We define the type of relational propositions, denoted rProp , as a binary relation over memories:

$$\text{rProp} := \text{memory} \rightarrow \text{memory} \rightarrow \text{Prop}$$

By convention, the first argument denotes the target memory m_t , and the second argument denotes the source memory m_s . An assertion $P \in \text{rProp}$ represents a predicate that expresses ownership over the memory footprints of both programs, specifying the contents of sub-heaps of the target and source.

Connective	Semantics	Description
$\ulcorner \psi \urcorner$	$\lambda m_t m_s. m_t = \emptyset \wedge m_s = \emptyset \wedge \psi$	Pure assertion
emp	$\lambda m_t m_s. m_t = \emptyset \wedge m_s = \emptyset$	Empty memory heaps
$P \wedge Q$	$\lambda m_t m_s. P(m_t, m_s) \wedge Q(m_t, m_s)$	Conjunction
$P \vee Q$	$\lambda m_t m_s. P(m_t, m_s) \vee Q(m_t, m_s)$	Disjunction
$P \rightarrow Q$	$\lambda m_t m_s. P(m_t, m_s) \implies Q(m_t, m_s)$	Implication
$\forall x, P(x)$	$\lambda m_t m_s. \forall x, P(x)(m_t, m_s)$	Universal quantification
$\exists x, P(x)$	$\lambda m_t m_s. \exists x, P(x)(m_t, m_s)$	Existential quantification
$P * Q$	$\lambda m_t m_s. \exists m_t^P m_t^Q m_s^P m_s^Q, m_t = m_t^P \uplus m_t^Q \wedge m_s = m_s^P \uplus m_s^Q$ $\wedge P(m_t^P, m_s^P) \wedge Q(m_t^Q, m_s^Q)$	Separating conjunction
$P \multimap Q$	$\lambda m_t m_s. \forall m'_t m'_s, P(m'_t, m'_s) \implies Q(m_t \uplus m'_t, m_s \uplus m'_s)$	Magic wand
$\Box P$	$\lambda m_t m_s. P(\emptyset, \emptyset)$	Persistent modality
$P \vdash Q$	$\forall m_t m_s, P(m_t, m_s) \implies Q(m_t, m_s)$	Logical entailment
$P \dashv\vdash Q$	$(P \vdash Q) \wedge (Q \vdash P)$	Logical equivalence

Figure 8: Base connectives of our relational separation logic.

The semantic definitions of our base logical connectives are summarized in ???. The entailment $P \vdash Q$ asserts that

whenever P holds on a memory pair, Q holds as well. Logical equivalence $P \dashv\vdash Q$ holds when both directions of entailment hold. The connectives satisfy the standard algebraic properties of separation logic:

- The separating conjunction $P * Q$ splits both the target heap m_t and the source heap m_s into disjoint sub-heaps³, assigning one part to P and the other part to Q . Because the sub-heaps must be strictly disjoint, this operator prevents duplicate resource ownership.
- The persistent modality $\Box P$ asserts that P holds on empty heaps. Because a persistent assertion requires no memory resources, it represents a duplicable fact: $\Box P \vdash \Box P * \Box P$.

Our memory logic is *linear* rather than affine. In an affine logic, heap chunks can be silently discarded via weakening: $P * Q \vdash P$. In our linear formulation, proving $P \vdash Q$ guarantees that every memory cell described by P is accounted for in Q . This strict resource discipline forbids any memory leak.

To preserve the interactive proof engineering benefits of the Iris Proof Mode (MoSeL) [krebbbers-18-mosel], we prove that our connectives satisfy the required typeclass hierarchy. This allows our formalization to use standard Iris tactics while executing directly on our shallow relational model.

4.2 Relational Assertions & Memory Predicates

Building on our memory model (??), we introduce primitive memory assertions for target and source states. We define relational points-to and freed assertions:

$$\begin{aligned} l \rightarrow_t v &:= \lambda m_t m_s. m_t = \{l \mapsto \text{Allocated}(v)\} \wedge m_s = \emptyset \\ l \rightarrow_s v &:= \lambda m_t m_s. m_t = \emptyset \wedge m_s = \{l \mapsto \text{Allocated}(v)\} \\ \text{free}_t(l) &:= \lambda m_t m_s. m_t = \{l \mapsto \text{Freed}\} \wedge m_s = \emptyset \\ \text{free}_s(l) &:= \lambda m_t m_s. m_t = \emptyset \wedge m_s = \{l \mapsto \text{Freed}\} \end{aligned}$$

The predicate $l \rightarrow_t v$ asserts that location l in the target heap contains value v , while requiring the source heap to be empty. Symmetrically, $l \rightarrow_s v$ asserts ownership of an allocated cell in the source heap. The predicates $\text{free}_t(l)$ and $\text{free}_s(l)$ assert ownership of deallocated memory cells in the target and source heaps, respectively.

Memory allocations in the target and source programs are not guaranteed to produce identical physical addresses. To account for address variations, we maintain an address mapping $I \subseteq \text{loc} \times \text{loc}$, where each pair $(l_t, l_s) \in I$ relates a target location l_t to a source location l_s . We enforce that I is a bijective mapping: target locations are uniquely related to source locations. This mapping I parameterizes the value similarity relation $v_t \bowtie_I v_s$ (??). As the bijection grows monotonically with fresh memory allocations, the value relation is preserved:

Lemma 3 (Monotonicity of $\bowtie_I$). If $I \subseteq I'$, then $v_t \bowtie_I v_s \implies v_t \bowtie_{I'} v_s$.

To relate entire target and source heaps across program execution, we adapt the memory injection predicate $\text{meminj}_E(I)$ defined by **gaher-22-simuliris**. When the set E is empty, this predicate owns all memory cells registered in I . It asserts that every active pair of locations $(l_t, l_s) \in I$ holds relationally similar values, and that deallocated cells are consistently marked as freed in both heaps.

When verifying an instruction that reads, writes, or deallocates a specific pair of locations $(l_t, l_s) \in I$, we need to access the underlying points-to or freed assertion. However, removing (l_t, l_s) from the mapping I would shrink I , breaking its monotonicity and invalidating the value similarity of existing pointers. To overcome this difficulty, we introduce the *exploited set* $E \subseteq I$. The set E tracks the address pairs whose individual heap assertions have been temporarily extracted from the global injection for local reasoning.

The interaction between the memory injection predicate, the bijection I , and the exploited set E is primarily governed by three lemmas:

- **Exploitation:** When $(l_t, l_s) \in I$ is unexploited, the assertion $\text{meminj}_E(I)$ can be opened to extract ownership of (l_t, l_s) while updating the exploited set to $E \cup \{(l_t, l_s)\}$.
- **Reintegration:** Once a memory operation completes, the updated points-to assertion $l_t \rightarrow_t v_t * l_s \rightarrow_s v_s$ (with $v_t \bowtie_I v_s$) or freed assertion $\text{free}_t(l_t) * \text{free}_s(l_s)$ can be released back into the injection, decreasing the exploited set to $E \setminus \{(l_t, l_s)\}$.

³Here $\uplus$ denotes the disjoint union of memory heaps, which is defined only when its operand domains are disjoint.

- **Extension:** When allocating fresh memory cells, the fresh pair of addresses (l_t, l_s) can be added to the memory injection provided their values are related by $\bowtie_I$.

4.3 Relational Hoare Quadruples

By combining the free simulation with relational separation logic, we define relational Hoare quadruples. In fixed target and source programs P_t and P_s , a relational Hoare quadruple relates a target function f_t to a source function f_s . The formal definition is given by:

$$\{P\} f_t \lesssim f_s \{Q\} := \Box \left(\forall \vec{v}_t \vec{v}_s, P(\vec{v}_t, \vec{v}_s) \multimap ([], \text{CallState}(f_t, \vec{v}_t)) \mathrel{\text{\tiny 0}\lesssim_0} ([], \text{CallState}(f_s, \vec{v}_s)) \{Q\} \right)$$

where P is a precondition relating the target and source argument vectors, and Q is a postcondition referring to the returned values. The simulation is here lifted to an assertion in rProp :

$$t \mathrel{j \lesssim_i} s \{ \Phi \} := \lambda m_t m_s. (t, m_t) \mathrel{j \lesssim_i} (s, m_s) \{ \Phi \}$$

The meaning of this quadruple is given by the free simulation: whenever the initial function arguments and heap states satisfy $P(\vec{v}_t, \vec{v}_s)$, if the source execution terminates without encountering undefined behavior, then the target execution also terminates with return values and memories satisfying $Q(v_t, v_s)$. If the source execution diverges, the target execution diverges as well. If the source execution reaches a stuck state, the simulation relation grants the compiler complete freedom.

The definition of the Hoare quadruple is guarded by a persistent modality $\Box$ to guarantee that the specification holds independently of heap ownership. As a consequence, a verified quadruple constitutes a duplicable resource that can be reused across multiple call sites throughout the program.

5 μRTL Proof Rules

Having established a simulation relation, the semantics of μRTL , and a separation logic over pairs of memories, we combine these components into a relational proof system. To facilitate the verification of code transformations, we establish a structured collection of relational proof rules operating directly on the simulation judgment $t \mathrel{j \lesssim_i} s \{ \Phi \}$. These rules are all proved sound with respect to the simulation relation and the μRTL semantics.

5.1 Asynchronous Rules

Our simulation relation decouples the execution of the target program from that of the source program. This separation yields asynchronous proof rules that advance one program while keeping the configuration of the other fixed.

Target proof rules express the conditions under which the target program can execute an instruction. The rules governing target execution of allocations and operations are displayed in ???. The source stepping rules mirror the target rules, operating on the source program counter, source registers, and source memory assertions.

$$\frac{\begin{array}{l} \text{eval_op}(\text{op}, \vec{v}) = \text{Some}(v) \quad \rho_t[\vec{r}] = \vec{v} \\ f_t[p_c] = \langle \text{dst} := \text{op } \vec{r} \rightarrow pc \rangle \quad (\sigma_t, \text{State}(f_t, pc, \rho_t[\text{dst} \mapsto v])) \mathrel{j' \lesssim_i} s_s \{ \Phi \} \end{array}}{(\sigma_t, \text{State}(f_t, pc_t, \rho_t)) \mathrel{j \lesssim_i} s_s \{ \Phi \}} \text{TARGET-OP}$$

$$\frac{\begin{array}{l} f_t[p_c] = \langle \text{dst} := \text{alloc}() \rightarrow pc \rangle \\ \forall l, v, l \rightarrow_t v \multimap (\sigma_t, \text{State}(f_t, pc, \rho_t[\text{dst} \mapsto \text{VPtr}(l)])) \mathrel{j' \lesssim_i} s_s \{ \Phi \} \end{array}}{(\sigma_t, \text{State}(f_t, pc_t, \rho_t)) \mathrel{j \lesssim_i} s_s \{ \Phi \}} \text{TARGET-ALLOC}$$

Figure 9: Asynchronous rules for allocation and operation instructions in the target program.

A key semantic difference between target and source execution appears in the allocation rule TARGET-ALLOC versus SOURCE-ALLOC. In our operational semantics, allocating a fresh memory cell non-deterministically initializes its

content with an arbitrary value. Because the target is demonic, the continuation in TARGET-ALLOC must hold universally for *all* possible fresh locations l and all initial values v . In contrast, because the source is angelic, when applying SOURCE-ALLOC, the value of the allocated memory cell may be chosen existentially:

$$\frac{v \in \text{val} \quad f_s[pc_s] = \langle dst := \text{alloc}() \rightarrow pc \rangle \quad \forall l, l \rightarrow_s v * s_t j \lesssim_{i'} (\sigma_s, \text{State}(f_s, pc, \rho_s[dst \mapsto \text{VPtr}(l)]) \{ \Phi \})}{s_t j \lesssim_i (\sigma_s, \text{State}(f_s, pc_s, \rho_s)) \{ \Phi \}} \text{SOURCE-ALLOC}$$

5.2 Undefined Behavior Exploitation

By the FSOURCESTUCK rule, our free simulation is skewed. When the source program reaches a configuration from which no further transition is possible without being in a final state, the simulation property $t j \lesssim_i s \{ \Phi \}$ holds directly. This property provides a powerful proof technique: whenever the source program is about to perform an operation, we can safely assume that the operation succeeds. If the operation were to fail, the source would be stuck, and the proof obligation would be instantly discharged. For instance, the exploit rule SOURCE-OP-EXPLOIT allows the proof to assume that the operation succeeds and produces a value v :

$$\frac{\rho_s[\vec{r}] = \vec{v} \quad f_s[pc_s] = \langle dst := \text{op } \vec{r} \rightarrow pc \rangle \quad \forall v, \vdash \text{eval_op}(\text{op}, \vec{v}) = \text{Some}(v)^\top * s_t j \lesssim_{1+i} (\sigma_s, \text{State}(f_s, pc, \rho_s[dst \mapsto v]) \{ \Phi \})}{s_t j \lesssim_i (\sigma_s, \text{State}(f_s, pc_s, \rho_s)) \{ \Phi \}} \text{SOURCE-OP-EXPLOIT}$$

5.3 Exploiting Memory Operations

In separation logic, memory resources cannot be obtained without explicit allocation. However, when programs manipulate values related through a memory injection $\text{meminj}_E(I)$, heap assertions can be extracted directly from the global injection invariant.

Suppose the source program is going to execute a load on an address addr_s that is related to a target address through $\text{addr}_t \bowtie_I \text{addr}_s$. Two deductions follow:

1. **Pointer validity:** If addr_s is not a valid pointer $\text{VPtr}(l_s)$, the source memory access fails and becomes stuck. Therefore, from the relation $\text{addr}_t \bowtie_I \text{addr}_s$, we deduce that $\text{addr}_s = \text{VPtr}(l_s)$ and $\text{addr}_t = \text{VPtr}(l_t)$ with $(l_t, l_s) \in I$.
2. **Active status:** Opening the memory injection $\text{meminj}_\emptyset(I)$ via the exploitation lemma reveals that the cell pair (l_t, l_s) is either actively allocated or already freed. However, if the location were freed, attempting a load in the source would produce undefined behavior. Thus, the freed case is instantly discharged, and the cell is guaranteed to be currently allocated.

Because we exploited the location pair $(l_t, l_s) \in I$, the set E in the returned memory injection is augmented. This reasoning is formalized in the rule SOURCE-LOAD-EXPLOIT shown in ???. Similar exploitation principles apply to the remaining memory operations.

$$\frac{f_s[pc_s] = \langle dst := !src \rightarrow pc \rangle \quad \rho_s[src] = \text{addr}_s \quad \text{addr}_t \bowtie_I \text{addr}_s \quad \text{meminj}_\emptyset(I) \quad \left(\begin{array}{l} \forall l_t l_s v_t v_s, \text{addr}_s = \text{VPtr}(l_s) \wedge \text{addr}_t = \text{VPtr}(l_t) \implies \\ l_t \rightarrow_t v_t * l_s \rightarrow_s v_s * \vdash v_t \bowtie_I v_s^\top * \text{meminj}_{\{(l_t, l_s)\}}(I) * \\ s_t j \lesssim_{i'} (\sigma_s, \text{State}(f_s, pc, \rho_s[dst \mapsto v_s]) \{ \Phi \}) \end{array} \right)}{s_t j \lesssim_i (\sigma_s, \text{State}(f_s, pc_s, \rho_s)) \{ \Phi \}} \text{SOURCE-LOAD-EXPLOIT}$$

Figure 10: Memory exploitation rule for source load instructions.

5.4 Synchronous Rules

While asynchronous and exploit rules provide flexibility for transformations that reorder or eliminate instructions, many compiler passes preserve the overall control flow of the program. For such passes, stepping target and

source programs independently introduces redundant bookkeeping. By composing source exploit rules with target stepping rules, we obtain synchronous rules that execute matching instructions in lockstep.

This is notably the case when reasoning about synchronous function calls. To ensure modularity, we avoid explicitly stepping through the execution of callee bodies. Instead, we rely on the Hoare quadruples introduced in ???. The synchronous call rule BOTH-CALL enables replacing a pair of matching function calls by their relational specification. Its structure highlights the core benefits of our relational separation logic.

Suppose both programs are about to execute a call:

- the target calls function f'_t with argument values $\vec{v}_t$, where $f_t[pc_t] = \langle dst_t := \text{call } f'_t(\vec{a}_t) \rightarrow pc'_t \rangle$ and $\rho_t[\vec{a}_t] = \vec{v}_t$;
- the source calls function f'_s with argument values $\vec{v}_s$, where $f_s[pc_s] = \langle dst_s := \text{call } f'_s(\vec{a}_s) \rightarrow pc'_s \rangle$ and $\rho_s[\vec{a}_s] = \vec{v}_s$;
- a relational specification $\{P\} f'_t \lesssim f'_s \{Q\}$ has already been established for the called functions.

Under these conditions, the BOTH-CALL rule proceeds as follows:

1. The caller must prove that current heap resources satisfy the relational precondition $P(\vec{v}_t, \vec{v}_s)$.
2. Instead of stepping through the internal instructions of f'_t and f'_s , the rule bypasses the callee bodies entirely.
3. Execution resumes at the return points pc'_t and pc'_s with the destination registers updated with return values, satisfying the postcondition $Q(v_t, v_s)$.

Because Hoare quadruples are enclosed in the persistent modality, they represent duplicable specifications that can be verified once per function, and reused across arbitrarily many call sites. This provides horizontal modularity: functions in a program can be compiled and verified independently, and their correctness guarantees compose cleanly through BOTH-CALL.

6 Case Studies

To evaluate the relational separation logic developed in this work, we present three case studies: a local transformation exploiting undefined behavior, a loop transformation hoisting an invariant load, and a control-flow optimization eliminating redundant jump chains.

6.1 Undefined Behavior Exploitation

We begin with a local program transformation that eliminates a redundant zero-check after an arithmetic division. Consider the source and target code fragments in ??.

Source Program		Target Program
1 : $r_1 := \text{Div } [r_n, r_d] \rightarrow 2$	$\implies$	1 : $r_1 := \text{Div } [r_n, r_d] \rightarrow 2$
2 : $r_b := \text{EqZ } [r_d] \rightarrow 3$		2 : $r_b := \text{ImmBool}(\text{false}) \rightarrow 3$
3 : ...		3 : ...

Figure 11: Exploiting division undefined behavior to simplify a zero test.

In the source program, instruction 1 divides the content of r_n by r_d , storing the quotient in r_1 . Instruction 2 tests whether the divisor r_d is zero via EqZ, storing the resulting boolean in r_b . In the target program, instruction 2 replaces this test with an immediate assignment of false.

The soundness of this transformation relies on the asymmetric nature of our simulation relation. When reasoning about instruction 1, we apply the rule SOURCE-OP-EXPLOIT. Because stuck source executions satisfy the simulation relation by FSOURCESTUCK, the rule provides the hypothesis that the operation succeeds: $\text{eval_op}(\text{Div}, [\rho_s(r_n), \rho_s(r_d)]) = \text{Some}(v)$. Under our operational semantics, division succeeds only when the

divisor $\rho_s(r_d) = \text{VInt}(z_d)$ satisfies $z_d \neq 0$. Assuming that target and source register banks agree initially, the target division executes identically, advancing both programs to $pc = 2$.

At program point 2, the source program tests register r_d for equality to zero. Because $\rho_s[r_d] = \text{VInt}(z_d)$ with $z_d \neq 0$, this evaluation deterministically yields false. In the target program, the immediate load executes via TARGET-OP and writes false into r_b without inspecting r_d . Both programs reach $pc = 3$ in matching register and memory states, establishing the soundness of this transformation.

6.2 Load Hoisting

We next examine a transformation hoisting a load out of a loop. ?? displays two versions of a factorial function $\text{fact}(n, \text{addr})$. The function initializes the memory cell at addr to 1, and computes the factorial of n .

Source Function (fact_bad)	Target Function (fact_good)
0 : $r_{res} := \text{ImmInt}(1) \rightarrow 1$	0 : $r_{res} := \text{ImmInt}(1) \rightarrow 1$
1 : $!r_{addr} := r_{res} \rightarrow 2$	1 : $!r_{addr} := r_{res} \rightarrow 2$
2 : $r_{cond} := \text{EqZ}[r_n] \rightarrow 3$	2 : $r_{one} := \text{ImmInt}(1) \rightarrow 3$
3 : if r_{cond} then 7 else 4	3 : $r_{cond} := \text{EqZ}[r_n] \rightarrow 4$
4 : $r_{res} := \text{Mul}[r_{res}, r_n] \rightarrow 5$	4 : if r_{cond} then 7 else 5
5 : $r_{one} := !r_{addr} \rightarrow 6$	5 : $r_{res} := \text{Mul}[r_{res}, r_n] \rightarrow 6$
6 : $r_n := \text{Sub}[r_n, r_{one}] \rightarrow 2$	6 : $r_n := \text{Sub}[r_n, r_{one}] \rightarrow 3$
7 : ret r_{res}	7 : ret r_{res}

Figure 12: Source and target factorial functions illustrating load hoisting.

In the unoptimized function fact_bad , the loop body performs a redundant heap load at $pc = 5$ on each iteration, reading the constant 1 stored at r_{addr} to decrement r_n . The optimized function fact_good hoists the load into the prelude. It further replaces the memory access with an immediate assignment of 1 into register r_{one} .

To prove the soundness of this transformation, we prove the following relational Hoare quadruple for all heap bijections I :

$$\{\text{same_args}_I\} \text{fact_good} \lesssim \text{fact_bad} \{\text{samer}_I\} \quad (6)$$

where same_args_I and samer_I relate input arguments and return values under an extendable heap bijection:

$$\begin{aligned} \text{same_args}_I(\vec{v}_t, \vec{v}_s) &:= \ulcorner \text{Forall2}(\bowtie_I, \vec{v}_t, \vec{v}_s) \urcorner * \text{meminj}_{\emptyset}(I) \\ \text{samer}_I(v_t, v_s) &:= \exists I', \ulcorner I \subseteq I' \urcorner * \ulcorner v_t \bowtie_{I'} v_s \urcorner * \text{meminj}_{\emptyset}(I') \end{aligned}$$

The precondition $\text{same_args}_I(\vec{v}_t, \vec{v}_s)$ asserts that input arguments are pairwise related under I , alongside an unexploited memory injection $\text{meminj}_{\emptyset}(I)$. The postcondition $\text{samer}_I(v_t, v_s)$ guarantees that return values remain related under a monotonically extended bijection $I' \supseteq I$, with the memory injection $\text{meminj}_{\emptyset}(I')$ fully restored. Allowing the bijection to grow monotonically accounts for any fresh memory allocations performed during execution.

The proof of ?? begins by stepping through the prelude. Both programs initialize r_{res} to 1. Applying SOURCE-STORE-EXPLOIT extracts (l_t, l_s) from $\text{meminj}_{\emptyset}(I)$, yielding $l_s \rightarrow_s \text{VInt}(1)$ and advancing the source to $pc_s = 2$. Applying TARGET-STORE then executes the target store, producing $l_t \rightarrow_t \text{VInt}(1)$. Finally, the target initializes r_{one} via TARGET-OP and reaches $pc_t = 3$, establishing the following loop invariant:

- The target program is executing fact_good at program counter $pc_t = 3$ with register bank ρ_t .
- The source program is executing fact_bad at program counter $pc_s = 2$ with register bank ρ_s .
- The register banks agree on the counter and accumulator: $\rho_t(r_n) \bowtie_I \rho_s(r_n)$ and $\rho_t(r_{res}) \bowtie_I \rho_s(r_{res})$.
- The registers r_{addr} hold related pointer values: $\rho_t(r_{addr}) = \text{VPtr}(l_t)$ and $\rho_s(r_{addr}) = \text{VPtr}(l_s)$ with $(l_t, l_s) \in I$.
- The target register r_{one} holds the constant $\text{VInt}(1)$ in ρ_t .

- The memory state satisfies $l_t \rightarrow_t \text{VInt}(1) * l_s \rightarrow_s \text{VInt}(1) * \text{meminj}_{\{(l_t, l_s)\}}(I)$, isolating the hoisted cell (l_t, l_s) while the remaining heap is governed by the injection.

The proof then proceeds by coinduction, evaluating the zero-test $\text{EqZ}(r_n) = b$ synchronously in both programs:

Base case ($b = \text{true}$): Both programs branch to the return instruction at $pc = 7$. The points-to assertions $l_t \rightarrow_t \text{VInt}(1) * l_s \rightarrow_s \text{VInt}(1)$ are returned to the memory injection via the reintegration lemma, restoring $\text{meminj}_{\emptyset}(I)$. Both programs execute $\text{ret } r_{res}$, satisfying the postcondition samer_I with $I' = I$.

Step case ($b = \text{false}$): Both programs execute the multiplication $r_{res} := r_{res} \times r_n$. At source $pc_s = 5$, the source load $r_{one} := !r_{addr}$ executes via SOURCE-LOAD directly from $l_s \rightarrow_s \text{VInt}(1)$ without requiring memory exploitation. Both programs then decrement $r_n := r_n - r_{one}$, preserving register agreement, and branch back to the loop headers ($pc_t = 3$ and $pc_s = 2$). This re-establishes the invariant, closing the case by the coinductive hypothesis.

6.3 Nop Tunneling

We now consider an optimization on arbitrary control-flow graphs: nop tunneling. Intermediate compiler passes frequently introduce redundant jump instructions ($\text{nop} \rightarrow pc$). A sequence of consecutive jumps forms a *nop chain*:

$$pc_0 \rightarrow pc_1 \rightarrow pc_2 \rightarrow \dots \rightarrow pc_k \rightarrow pc_{\text{final}}$$

where each node pc_m ($0 \leq m < k$) contains $\text{nop} \rightarrow pc_{m+1}$, and pc_{final} is the first non-nop instruction. Nop tunneling short-circuits these chains by redirecting branch targets directly to pc_{final} , reducing instruction count and eliminating superfluous jump overhead.

The transformation is defined in Rocq over an arbitrary μRTL function f . To ensure termination, even in the presence of cyclic jump chains (such as an infinite loop $pc: \text{nop} \rightarrow pc$), the computation of the final successor of an instruction at pc is parameterized by a recursion depth $d \in \mathbb{N}$:

$$\begin{aligned} \text{find_next_pc}(0, f, pc) &:= pc \\ \text{find_next_pc}(1 + d, f, pc) &:= \begin{cases} \text{find_next_pc}(d, f, pc') & \text{if } f[pc] = \langle \text{nop} \rightarrow pc' \rangle \\ pc & \text{otherwise} \end{cases} \end{aligned}$$

The optimization $\text{tunnel_nop}(d, f)$ maps find_next_pc over every instruction in the control-flow graph of f , redirecting each successor label pc to $\text{find_next_pc}(d, f, pc)$. The function entry point is updated in the same manner. To establish the soundness of nop tunneling for an arbitrary function f , we prove the following relational Hoare quadruple for all depths d and heap bijections I :

$$\{\text{same_args}_I\} \text{tunnel_nop}(d, f) \lesssim f \{\text{samer}_I\} \quad (7)$$

The proof relies on a coinductive simulation invariant. Given a fixed initial fuel depth , the invariant asserts that at any step of execution:

- The target program is executing the optimized function $\text{tunnel_nop}(\text{depth}, f)$ at program counter pc_t with register bank ρ_t .
- The source program is executing the original function f at program counter pc_s with register bank ρ_s .
- The target program counter is the tunneled successor of the source program counter with remaining fuel d : $pc_t = \text{find_next_pc}(d, f, pc_s)$.
- The register banks agree on all registers: $\forall r, \rho_t(r) \bowtie_I \rho_s(r)$.
- The memory heaps are fully synchronized under a monotone bijection $I: \text{meminj}_{\emptyset}(I)$.

The proof of ?? proceeds by coinduction followed by an inner induction on the remaining fuel d :

Base case ($d = 0$): When fuel is exhausted, $\text{find_next_pc}(0, f, pc_s) = pc_s$, so $pc_t = pc_s$. The target and source are positioned at identical program points. By case analysis on the instruction at pc_s , both programs progress in lockstep for each instruction kind. The next target program counter is the redirected label $\text{find_next_pc}(\text{depth}, f, pc'_s)$, which re-establishes the invariant with full fuel $d = \text{depth}$. The coinductive hypothesis closes this case.

653 **Inductive step** ($d > 0$): We inspect the source instruction at pc_s :

- 654 • If pc_s maps to a jump $\text{nop} \rightarrow pc'_s$, then $\text{find_next_pc}(1 + d', f, pc_s) = \text{find_next_pc}(d', f, pc'_s)$. The
655 source executes the jump via **SOURCE-NOP**, advancing to pc'_s . The target remains at pc_t . The inner
656 induction hypothesis on d' concludes this case.
- 657 • If pc_s does not refer to a jump instruction, $\text{find_next_pc}(1 + d', f, pc_s) = pc_s$, so $pc_t = pc_s$. As in
658 the base case, target and source progress in lockstep, resetting fuel to $depth$ for successors. The
659 coinductive hypothesis closes this case.

660 When verifying function call instructions, the proof applies the synchronous **BOTH-CALL** rule. Because the
661 coinductive invariant is parameterized over arbitrary μRTL functions, it can be instantiated directly for the callee.
662 Therefore, the relational Hoare quadruple required by **BOTH-CALL** is discharged without additional lemmas. This
663 illustrates the modularity of our relational separation logic: function-level optimizations are verified in isolation
664 and compose cleanly across whole-program call graphs.

665 7 Related Work

666 We compare our approach with existing verified compilers, simulation techniques, and relational separation
667 logics.

668 7.1 Verified Compilers

669 **Non-Determinism** CompCert [leroy-09-compcert] is a verified C compiler that establishes correctness by
670 proving forward simulation diagrams for each compilation phase. When the target language is deterministic, a
671 forward simulation implies a backward simulation, which guarantees semantic preservation. However, concurrent
672 extensions such as CompCertTSO [sevcik-11-compcerttso] introduce non-determinism into target executions,
673 requiring backward simulations to be established directly. CakeML [kumar-14-cakeml, tan-19-cakeml] avoids
674 this complication by enforcing determinism throughout its source semantics and intermediate representations,
675 disallowing concurrency and internal non-deterministic choice. In our framework, we directly establish a safe
676 backward simulation. This enables our system to accommodate internal non-determinism in the target language.
677 Nevertheless, our current μRTL operational semantics features only limited non-determinism, specifically fresh
678 memory allocation, with no support for concurrent primitives.

679 **Stuttering** CompCert handles stuttering by indexing simulation relations with an explicit well-founded order,
680 which must strictly decrease on stuttering steps. CakeML uses a fuel-indexed operational semantics. Because
681 evaluation in CakeML is deterministic, semantic preservation is established by case analysis on the existence of a
682 sufficient fuel value to reach termination. In our approach, we address stuttering coinductively through a free
683 simulation relation [cho-23-stuttering]. This formulation avoids constructing explicit well-founded metrics or
684 managing fuel bounds.

685 **Memory Models** CompCert uses a block-based memory model [leroy-08-memory], in which program
686 transformations that alter memory layouts are justified through global memory injections. These injections
687 map source memory blocks to target blocks across the global heap. CakeML similarly tracks memory through a
688 global mutable store, lowering this store to a flat linear array in the backend. In our work, we employ separation
689 logic to reason locally about memory fragments and memory injection. Local heap assertions avoid reasoning
690 globally about the entire memory state. We hope that this formulation will ease future proofs of transformations
691 impacting memory.

692 **Modularity** CompCert is structured as a whole-program compiler with global call-stack invariants. To support
693 verified separate compilation and modular linking, several extensions have been developed [stewart-15-compcomp,
694 kang-16-light]. Similarly, while early CakeML proofs were whole-program, recent extensions support dynamic
695 compilation primitives [sewell-23-cakes]. In our framework, relational Hoare quadruples provide horizontal
696 compositionality, allowing verified code components to be used independently. However, our current formaliza-
697 tion operates within a single intermediate language, leaving compilation passes between distinct languages to
698 future work.

699 7.2 Relational Separation Logics

700 **Logical Foundations** Simuliris [gaher-22-simuliris] is a relational separation logic framework implemented
701 within Iris [jung-18-iris]. Recent work on Tail Modulo Cons (TMC) by allain-25-tmc similarly embeds a rela-
702 tional separation logic in Iris to verify the soundness of the TMC transformation for a functional language. Other
703 relational frameworks built on Iris include ReLoC [frumin-21-reloc] for contextual refinement in concurrent
704 languages, and Melocoton [gueneau-23-melocoton] for verified interoperability between OCaml and C.

705 While Iris provides substantial expressive power, its semantic model relies on complex constructions, such
706 as step-indexed resource algebras and modal update operators. In our framework, we do not build upon the
707 semantic model of Iris. Instead, we formulate our relational separation logic directly over concrete heap states
708 and coinductive simulation relations. Our logic definitions remain direct while maintaining compatibility with
709 MoSeL [krebbbers-18-mosel], the interactive separation logic proof mode.

710 **Program Representation and Control Flow** The nature of the underlying languages also distinguishes our
711 work from prior relational separation logics. Both Simuliris and TMC target high-level languages with structured,
712 syntax-directed control flow. In contrast, our framework supports μ RTL, a low-level register transfer language
713 where programs are represented as control-flow graphs. This requires relational proof rules capable of handling
714 unstructured control flow.

715 **Verified Transformations** The scope of verified program transformations also differs. Simuliris focuses on
716 fine-grained concurrent optimizations, such as code motion and instruction reordering under weak memory
717 models. TMC proves the correctness of a specialized source-to-source transformation, converting non-tail-
718 recursive functions into tail-recursive loops by mutating newly allocated constructor fields. In our work, we
719 evaluate our logic on backend compiler transformations over control-flow graphs, verifying optimizations that
720 eliminate redundant jump chains.

722 A Coinduction Example: Stream Bisimilarity

723 To illustrate how coinductive reasoning operates in practice, consider the *bisimilarity* relation $\approx$ on streams of
 724 natural numbers, $\text{Stream}(\mathbb{N})$, defined coinductively by the following inference rule:

$$\frac{n_1 = n_2 \quad s_1 \approx s_2}{\text{scons}(n_1, s_1) \approx \text{scons}(n_2, s_2)} \text{ Bisim-Step}$$

725 Let us prove that this relation is reflexive:

$$\forall s \in \text{Stream}(\mathbb{N}), s \approx s.$$

726 The proof proceeds by coinduction on the *bisimilarity* relation:

727 1. Applying the coinduction principle provides the coinductive hypothesis:

$$\text{ColH} : \forall s' \in \text{Stream}(\mathbb{N}), s' \approx s',$$

728 with the restriction that ColH may only be applied under a constructor of the coinductive relation.

729 2. We decompose the stream s into a head $n \in \mathbb{N}$ and a tail $s' \in \text{Stream}(\mathbb{N})$, so that $s = \text{scons}(n, s')$. The
 730 goal becomes:

$$\text{scons}(n, s') \approx \text{scons}(n, s').$$

731 3. We apply the coinductive rule Bisim-Step. The first premise $n = n$ holds trivially, leaving the second
 732 premise $s' \approx s'$ to be established.

733 4. Because the constructor Bisim-Step has been applied, we can now legitimately invoke ColH. This immedi-
 734 ately concludes the proof.

735 B Free Simulation Example

736 To illustrate how the free simulation operates in practice, we consider a small imperative language with commands
 737 `skip` | $c_1; c_2$ | `while` $b \{c\}$ | `ret` v , where `ret` v is a terminal configuration. The small-step reduction relation
 738 $c \rightarrow c'$ is defined by:

$$\begin{array}{ll} \text{skip}; c \rightarrow c & \\ \text{while } b \{c\}; k \rightarrow c; \text{while } b \{c\}; k & (\text{if } b = \text{true}) \\ \text{while } b \{c\}; k \rightarrow k & (\text{if } b = \text{false}) \end{array}$$

739 We instantiate the index domains with the natural numbers, $I = J = (\mathbb{N}, <)$, and define the target and source
 740 programs:

$$\begin{array}{l} P_t := \text{while } (b) \{ \text{skip} \}; \text{ret } () \\ P_s := \text{while } (b) \{ \text{skip}; \text{skip}; \text{skip} \}; \text{ret } () \end{array}$$

741 We wish to prove that $P_t \approx_0 P_s \{=\}$. We proceed by coinduction on the free simulation. This gives us the
 742 following coinductive hypothesis:

$$\forall j > 0, \forall i > 0, \quad P_t \approx_{j \lesssim i} P_s \{=\}.$$

743 We then perform case analysis on the boolean condition b :

- 744 • If $b = \text{false}$, both programs exit their loops in a single step: $P_t \rightarrow \text{ret } ()$ and $P_s \rightarrow \text{ret } ()$. Applying
 745 FTARGETSTEPS and FSOURCESTEPS, both configurations reach `ret` $()$, which are final values returning $()$.
 746 The goal is thus closed immediately by FRELATED.

- If $b = \text{true}$, using FTARGETSTEPS the target loop unrolls: $P_t \rightarrow \text{skip}; P_t$, and the target index is updated to $j = 1$. Again with FTARGETSTEPS , the target reduces: $\text{skip}; P_t \rightarrow P_t$, and the target index is increased to $j = 2$. The simulation goal becomes $P_t \lesssim_0 P_s \{=\}$.

The source loop then executes four steps: first, $P_s \rightarrow \text{skip}; \text{skip}; \text{skip}; P_s$, updating the source index to $i = 1$ via FSOURCESTEPS ; next, three successive applications of FSOURCESTEPS consume the three `skip` statements, yielding index $i = 4$ and state P_s . The simulation goal becomes $P_t \lesssim_4 P_s \{=\}$.

Having accumulated positive progress indices ($0 < 2$ and $0 < 4$), we can apply the coinductive hypothesis to conclude the proof.

The complete derivation tree for the loop iteration case ($b = \text{true}$) is displayed in ??.

$$\begin{array}{c}
\frac{0 < 2 \quad 0 < 4}{P_t \lesssim_4 P_s \{=\}} \text{CIH} \\
\frac{\frac{P_t \lesssim_4 P_s \{=\}}{P_t \lesssim_1 \text{skip}; \text{skip}; \text{skip}; P_s \{=\}} \text{FSOURCESTEPS } (\times 3)}{\frac{P_t \lesssim_0 P_s \{=\}}{\text{skip}; P_t \lesssim_0 P_s \{=\}} \text{FTARGETSTEPS}} \text{FSOURCESTEPS} \\
\frac{\text{skip}; P_t \lesssim_0 P_s \{=\}}{P_t \lesssim_0 P_s \{=\}} \text{FTARGETSTEPS}
\end{array}$$

Figure 13: Derivation tree for the loop iteration case ($b = \text{true}$).

As illustrated by this example, the source and target programs can execute arbitrary numbers of asynchronous steps, at whatever pace is convenient, until matching invariants are reached.